

D.3 Dynamic Submodular

The standard submodular function is mapping $2^{[n]}$ to real. Here we need a more general definition that maps $2^{[n]} \times 2^{[n]}$ to real.

Definition D.3 (Strong submodular). *We define function $F : 2^{[n]} \times 2^{[n]} \rightarrow \mathbb{R}$, then for any set $Z \subset [n]$, we assume that $F(Z, \cdot) : 2^{[n]} \rightarrow \mathbb{R}$ is a submodular function.*

In fact, the above definition is stronger than we want, what we really need can be written as follows. We remark that in Definition 4.1 we provide an informal definition. Here we provide a more detailed version of the definition.

Definition D.4 (Dynamic submodular framework, formal version of Definition 4.1). *Define function*

$$F : 2^{[n]} \times [n] \times 2^{[n]} \rightarrow \mathbb{R}.$$

Then for any index $i \in [n]$, any set $Z \subseteq [i-1]$, we assume that

$$F(Z, i, \cdot) : 2^{[n]} \rightarrow \mathbb{R}$$

is a submodular function w.r.t. to Z , i.e.,

- *For all sets $X, Y \subset [n]$ satisfy that $Z \subset X \subset Y$,*
- *For all element $x \in [n]$ satisfy that $x \in [n] \setminus Y$,*

we have

$$f_{Z,i}(X \cup \{x\}) - f_{Z,i}(X) \geq f_{Z,i}(Y \cup \{x\}) - f_{Z,i}(Y),$$

where $f_{Z,i}(\cdot) := F(Z, i, \cdot)$.

Remark D.5. *We remark that Definition D.4 is a weaker version of Definition D.3. We also like to mention that the informal definition (see Definition 4.1) only contains two input parameters, but in fact we need three input parameters (see Definition D.4).*

In the later, when we use $f_{Z,i}(\cdot)$, we will replace Z by S_i for convenient of analysis, for example see Definition D.23, Definition D.24, Definition D.25 and Definition D.26.

D.4 Static Attention

Before we describe the recursive attention computation, we will first describe the static version of attention computation as follows (for examples, see Definition 1.1 in [98] and others [97, 100, 101, 106, 103, 118, 108]):

Definition D.6. *Given three matrices $Q, K, V \in \mathbb{R}^{d \times d}$, the goal is to compute*

$$\text{Att}(Q, K, V) := D^{-1} A \cdot V$$

where square matrix $A \in \mathbb{R}^{n \times n}$ can be rewritten as follows

$$A = \exp(QK^\top)$$

and diagonal matrix $D \in \mathbb{R}^{n \times n}$ can be written as follows

$$D = \text{diag}(A \mathbf{1}_n)$$

Here we apply $\exp()$ to a matrix entry-wisely to a matrix. $\mathbf{1}_n$ is a length- n vector where all the entries are ones. The operator $\text{diag}()$ is turning a vector into a diagonal matrix.

D.5 Recursive Attention Definition

We first provide some definitions.

Definition D.7. *Given a query matrix $Q \in \mathbb{R}^{n \times d}$, we use $Q_{i,*}$ to denote a length- d row vector that represents the i -th row of Q for each $i \in [n]$.*

Definition D.8. *Given a key matrix $K \in \mathbb{R}^{n \times d}$, we use $K_{\leq i,*}$ to denote a $i \times d$ matrix that selects the first i rows from K .*

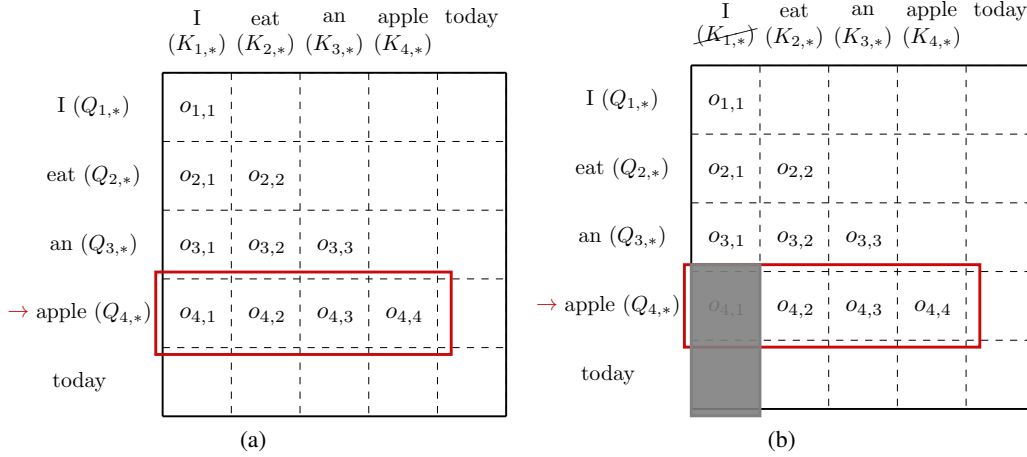


Figure 14: (a) Exact version of the attention computation. Here, our example is about the language modeling task. At this stage, the model predicts the word 'apple' and computes the exact attention vector o_4 . (b) Approximate version of the attention computation. Let the budget of the number of tokens we can track be 3. We truncate the key matrix for the first token $K_{1,*}$ and only keep $K_{2,*}$, $K_{3,*}$ and $K_{4,*}$ in the memory. Compared with the exact version, we don't compute $o_{4,1}$.

Next, we show the exact version of computing attention.

Definition D.9 (Exact Version). *Given $Q, K \in \mathbb{R}^{n \times d}$*

- For each $i \in [n]$, we use $o_i \in \mathbb{R}^i$ to denote a length- i vector.
- For each $j \in [i]$, we use $o_{i,j}$ to denote the j -th coordinate of vector $o_i \in \mathbb{R}^i$

For the first layer, we have

- $o_{1,1} = 1$.

For the second layer, we have

- length-2 vector $o_2 := \underbrace{D_2^{-1}}_{\text{scalar}} \cdot \exp(\underbrace{Q_{2,*}}_{1 \times d} \underbrace{(K_{\leq 2,*})^\top}_{d \times 2})$
- scalar $D_2 := \underbrace{\exp(Q_{2,*}(K_{\leq 2,*})^\top)}_{1 \times 2} \cdot \underbrace{\mathbf{1}_2}_{2 \times 1}$

For each $i \in [n]$, for the i -th layer, we have

- length- i vector $o_i := \underbrace{D_i^{-1}}_{\text{scalar}} \cdot \exp(\underbrace{Q_{i,*}}_{1 \times d} \underbrace{(K_{\leq i,*})^\top}_{d \times i})$
- scalar $D_i := \underbrace{\exp(Q_{i,*}(K_{\leq i,*})^\top)}_{1 \times i} \cdot \underbrace{\mathbf{1}_i}_{i \times 1}$

Now, we show the approximate version of computing attention. Instead of computing the entire attention o_i , we only compute the attention of the tokens that are being tracked.

Definition D.10 (Approximate Version). *Given $Q, K \in \mathbb{R}^{n \times d}$*

- For each $i \in [n]$, we use $o_i \in \mathbb{R}^i$ to denote a length- i vector.
- For each $j \in [i]$, we use $o_{i,j}$ to denote the j -th coordinate of vector $o_i \in \mathbb{R}^i$
- Let $k \in [n]$ be the budget of the number of tokens we can track (due to the memory issue).

For the first layer, we have

- $o_{1,1} = 1$.

For the second layer, we have

- length-2 vector $o_2 := \underbrace{D_2^{-1}}_{\text{scalar}} \cdot \exp(\underbrace{Q_{2,*}}_{1 \times d} \underbrace{(K_{\leq 2,*})^\top}_{d \times 2})$
- scalar $D_2 := \underbrace{\exp(Q_{2,*}(K_{\leq 2,*})^\top)}_{1 \times 2} \cdot \underbrace{\mathbf{1}_2}_{2 \times 1}$

For each $i \in [n]$, for the i -th token, we have

- Let $S_i \subset [n]$ denote the tokens we're tracking when we are predicting the i -th token.
- $|S_i| = k$
- $|S_i \setminus S_{i-1}| \leq 1$ or equivalently $|S_i \cap S_{i-1}| \geq k - 1$
- length- i vector $o_i := \underbrace{D_i^{-1}}_{\text{scalar}} \cdot \exp(\underbrace{Q_{i,*}}_{1 \times d} \underbrace{(K_{S_i,*})^\top}_{d \times i})$
- scalar $D_i := \underbrace{(\exp(Q_{i,*}(K_{S_i,*})^\top) - 1_{[i] \setminus S_i})}_{1 \times i} \cdot \underbrace{\mathbf{1}_i}_{i \times 1}$

In a certain sense, the above definition is related to finding heavy hitters in compressive sensing (for more background on compressive sensing, we refer readers to [124, 125, 126]).

D.6 Eviction Policy

The goal of this section is to our eviction policy under space limitation. We start by giving a process without having space limitations. We denote the attention query matrix as $Q \in \mathbb{R}^{n \times d}$ and the key matrix as $K \in \mathbb{R}^{n \times d}$. $Q_{i,*}$ represents the i -th row of Q and $K_{\leq i,*}$ represents the first i rows of K . For simplicity, $K_{S_i,*}$ denotes a sub-matrix of K which selects S_i rows from K .

Definition D.11 (The generative process with Full knowledge). *For each $i \in [n]$, for the i -th token, we have*

- length- i vector $o_i := D_i^{-1} \cdot \exp(Q_{i,*}(K_{\leq i,*})^\top)$, which denotes the attention of the i -th word.
- scalar $D_i := \exp(Q_{i,*}(K_{\leq i,*})^\top) \cdot \mathbf{1}_i$, which denotes the normalization factor.

In the above process, since we have no space limitation, then we can keep all the scores. In the following process, we show how to handle things when there is a space limitation. In this case, we need to dynamically maintain set S_i such that $|S_i| \leq k$ (Compared to the full knowledge case, we can think of $|S_i| = i$).

Definition D.12 (H₂O Eviction Policy, Formal version of Definition 4.3). *Let k denote the budget of space and $k < n$. Let $F_{\text{score}} : 2^{[n]} \rightarrow \mathbb{R}$ denote certain score function. Let S_{i-1} denote the source set. Let S_i denote the target set. We defined the eviction policy $g : S_{i-1} \rightarrow S_i$ s.t.*

- $|S_i| \leq k$ (KV cache size is not changing over the time)
- $|S_i \setminus S_{i-1}| \leq 1$ (we can evict at most 1 KV in the KV cache)
- We construct $S_i \leftarrow (S_{i-1} \cup \{i\}) \setminus \{u\}$ as $u \leftarrow \arg \max_{v \in (S_{i-1} \cup \{i\})} F_{\text{score}}(S_{i-1} \cup \{i\} \setminus \{v\})$

Remark D.13. We remake that, in Definition 4.3, we introduce a simplified notation where we state that $|S_i| = k$ in the first bullet point, but in general, we consider the case where $|S_i| \leq k$ for the first k tokens. To accommodate this more general scenario, we present Definition D.12, which provides a broader definition that handles the situation where $|S_i| \leq k$.